

Understanding Polymorphism

- **Polymorphism** is an important concept in Object-Oriented Programming (OOP).
- The word Polymorphism comes from two words:

Poly = Many

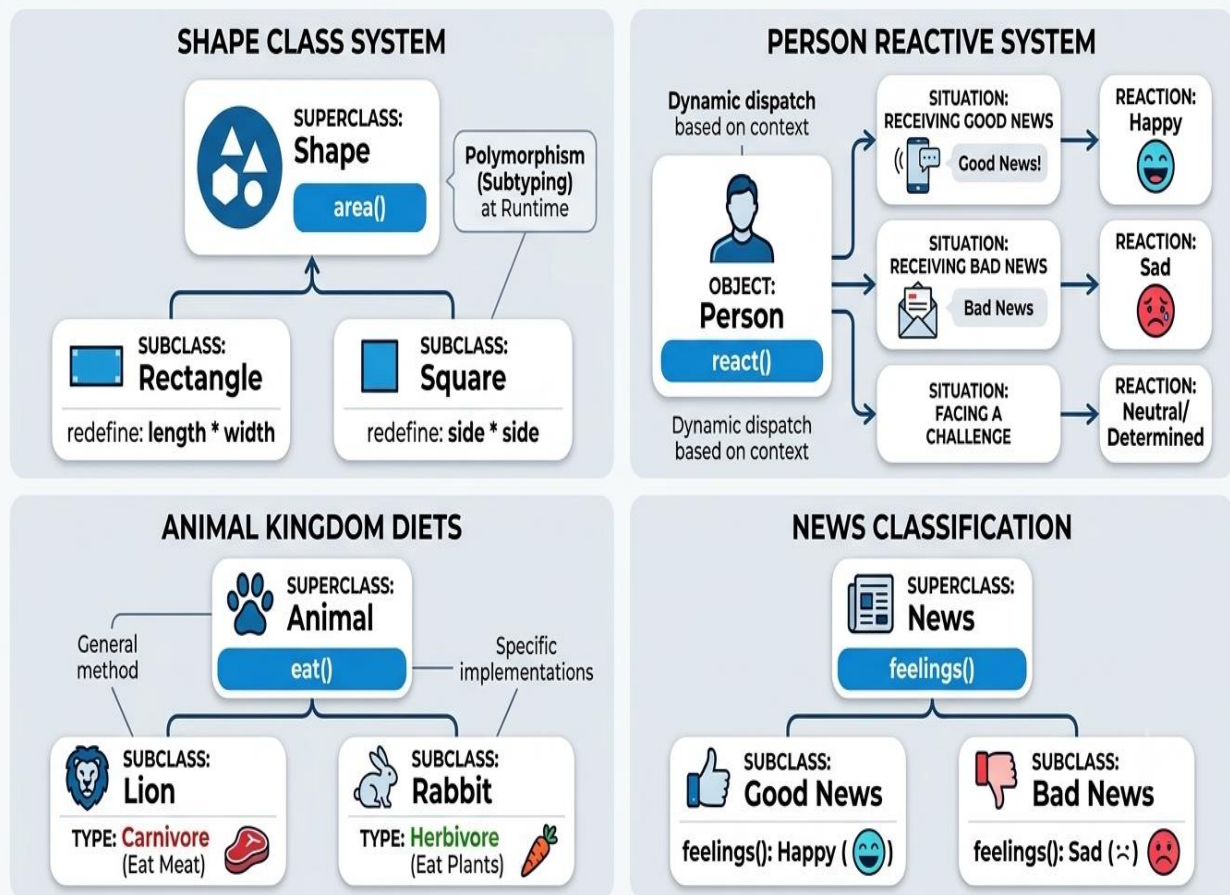
Morph = Forms

So, Polymorphism means “**one thing having many forms.**”

- Polymorphism means one thing having many forms. **(Simple definition)**
(A **person** can have many roles. The same person can be a **student at college, an employee at a company, and a customer in a shop.**)
- Polymorphism means the same thing can behave differently in different situations. **(Mobile related definition)**
(A **mobile phone** can be used for **calling, taking photos, watching videos, or playing games.** The same phone performs different activities depending on the situation.)
- Polymorphism means one common operation can have different behaviours. **(Programming-oriented definition)**
(Consider the operation “**make a sound.**”
A **dog barks**, a **cat meows**, and a **cow moos**. The operation is the same—making a sound—but each animal behaves differently.)
- Polymorphism means different objects can respond differently to the same action. **(Parent-child relationship definition)**
(If we tell different animals to “**eat,**” a **lion eats meat**, a **cow eats grass**, and a **rabbit eats vegetables.**
The action is the same—**eat**—but the behavior is different.)
- Polymorphism means one name or action can represent different behaviors. **(Simple classroom definition)**
(The word “**draw**” can mean different things depending on the object. You can **draw a circle, draw a line, or draw a picture.**)

- Polymorphism means the same person, thing, or action can take different forms depending on the situation. (**Real-life definition**)
(A **teacher** can behave differently in different situations. In the classroom, the teacher **teaches students**; at home, the same person may be a **parent**; while shopping, the person becomes a **customer**.)
- Polymorphism means the same action produces different results depending on who or what performs it. (**Behavior-based definition**)
(When you press the “**play**” button:
On a music app → a **song plays**
On a video app → a **video plays**
In a game → the **game starts**
The action is the same—**play**—but the result is different.)

EXPLORING CLASSIC POLYMORPHISM: ONE CONCEPT, MANY FORMS



Method Hiding in Java

- **Method hiding** occurs when a **child class defines a static method with the same name and signature as a static method in the parent class.**
- Method hiding means a child class hides the parent's static method instead of overriding it.

Real-life example (Method Hiding)

Imagine a company has a **general announcement system**.

- The **Company** has a static announcement: "Company working hours are 9 AM to 6 PM."
- A **Branch** has its own static announcement with the same name: "Branch working hours are 10 AM to 7 PM."

(The branch doesn't **override** the company's announcement dynamically. It **hides** it with its own version.)

Java Example :-

(Parent.java)

```
class Parent {  
    // Static method  
    static void display() {  
        System.out.println("Parent display()");  
    }  
}
```

(Child.java)

```
class Child extends Parent {  
    // Method Hiding  
    static void display() {  
        System.out.println("Child display()");  
    }  
}
```

(MethodDemo.java)[main class]

```
public class MethodDemo {  
    public static void main(String[] args) {  
        Parent p = new Child();  
        // Method Hiding  
        p.display();  
    }  
}
```

(_Out put)

Parent display()

Execution Flow of This program :(step by step)

1. JVM starts MethodDemo.main()



2. Executes:

Parent p = new Child();



3. A Child object is created in **Heap memory**.



4. Reference variable `p` is created.

Reference type = Parent

Actual object = Child



5. Executes:

p.display();



6. `display()` is static.



7. Static methods are resolved using
the reference type, not the object type.



8. Reference type is `Parent`.



9. Therefore:

Parent.display() is called.



10. Output:

Parent display()

Here, Why is Child Method not execute ?

Ans)

Child.display() is not executed because **display()** is a static method. Static methods are resolved at compile time based on the reference type, not at runtime based on the actual object.

In **Parent p = new Child();**, the reference type is **Parent**, so when we call **p.display()**, Java resolves it to **Parent.display()**.

This is called method hiding, not method overriding.